

v8.10

Programación 1 - Trabajo Práctico final

2026

Debe resolver los ejercicios y responder a las preguntas de este trabajo en la plantilla dada como parte del enunciado ApellidoNombre-Final.rkt

1 Códigos de errores

Es un hecho conocido que toda la información que se transmite entre máquinas y se almacena en computadoras se codifica con ceros y unos. Imágenes, texto e inclusive el mismo código que resuelve este trabajo práctico se almacena en forma de ceros y unos en la computadora de algún/a estudiante dispuesto/a a resolverlo. Si a estas imágenes, texto o códigos los almacenamos de forma cruda, es decir, sin utilizar más memoria que la que ocupan, lo más probable es que con el tiempo algunos bits cambien debido a las condiciones físicas en las que se encuentran las componentes electrónicas. Algo parecido ocurre con la transmisión de información por medios físicos: las imágenes, el texto o el código es codificado en ceros y unos, luego estos se transmiten como pulsos, haces de luz u ondas a través de cables, fibras o antenas y finalmente se interpretan como ceros y unos en el receptor. Las conversiones entre lo digital y lo analógico y la transmisión por el medio físico pueden equivocar bits en el mensaje y por lo tanto corromperlo.

Una forma usual de garantizar correctitud para un bloque de ceros y unos es utilizar más de una copia del bloque y luego utilizar algún algoritmo para desempatar y determinar cuál es el bloque correcto a partir de sus bits. Supongamos que usamos 3 copias de un bloque para evitar perder bits en una transmisión.

Por un lado, si un bit cambia en un solo bloque entonces podremos determinar cuál es el correcto porque las otras dos copias lo contienen y ganan por mayoría. Ahora si se produce el mismo error en 2 copias diferentes gana el bit erróneo en el algoritmo de desempate y terminamos con un bloque corrupto.

Por otro lado, al utilizar 3 copias sólo $\frac{1}{3}$ del espacio se utiliza para los datos mientras que $\frac{2}{3}$ del espacio es información extra que utilizamos para poder detectar y recuperar errores. A este espacio extra que se utiliza para mejorar la correctitud de los mensajes que se transmiten o almacenan se llama **redundancia**. En general el verbo redundar se utiliza para indicar que ciertos datos se puede deducir a partir de otros y no necesariamente cuando un dato es copia literal de otro: si A se puede calcular a partir de B entonces A es redundante.

En el caso que estudiaremos en este trabajo, los códigos de Hamming, los bits redundantes serán bits inteligentemente elegidos para utilizar la mínima cantidad de espacio extra y a la vez poder detectar con exactitud dónde se produce un error (y felizmente en algunos casos poder corregirlo).

2 Bits de paridad

La técnica de corrección de errores de Hamming se basa en una idea muy primitiva de corrección de errores que es la del bit de paridad. Un bit de paridad es un cero o un uno agregado a un mensaje que determina si la cantidad de unos en el mensaje es par o impar. Si la cantidad de unos en el mensaje es par entonces su paridad es 0 y si es impar la paridad del mensaje es 1.

Ejercicio 1. Diseñe una función que dada una lista de ceros y unos (un mensaje) devuelva un 1 si la cantidad de unos en la lista es impar y 0 en caso contrario.

```
(check-expect (paridad (list 1 1 0 0 1 0)) 1)
(check-expect (paridad (list 1 0 0 0 1 0)) 0)
```

Luego de resolver el ejercicio sin patrones de alto orden puede intentar resolverlo usándolos.

Analicemos por qué es útil el bit de paridad y cuáles son sus limitaciones.

Para enviar el mensaje `(list 1 1 0 1)` construimos una nueva lista con un bit redundante a la cabeza que funciona de bit de paridad `(cons 1 (list 1 1 0 1))`. A este listado de bits lo llamamos **bloque** para distinguirlo del mensaje.

En el receptor puede ocurrir que

- llegue el bloque `(cons 1 (list 1 1 0 1))` el cual contiene el mismo mensaje enviado por el emisor y el bit de paridad calculado correctamente.
- llegue el mensaje `(cons 1 (list 1 0 0 1))`. Vemos que el bit de paridad no se corresponde con la paridad del mensaje con lo cual detectamos que hay por lo menos un error en el mensaje. Si suponemos que se produjo exactamente un error no podemos distinguir si se cambió un cero por un uno (y cuál cero se cambió) o si se cambió un uno por un cero (y cuál uno se cambió).
- llegue el mensaje `(cons 1 (list 1 0 0 0))` el cual no contiene el mensaje del emisor pero tiene el bit de paridad correcto y coincidente con el bit de paridad del emisor. Esto se considera un falso negativo pues el bit de paridad no descubre el error.
- llegue el mensaje `(cons 0 (list 1 1 0 1))` el cual contiene el mensaje del emisor pero tiene un error en el bit de paridad (la corrupción se produjo en el bit de paridad). Esto se considera un falso positivo pues el bit de paridad descubre un error que no es tal.

Vemos que utilizar un bit de paridad puede ser bueno para la detección de errores pero no para su corrección y que además puede producir falsos positivos y falsos negativos. Sin embargo es increíblemente modesto con el espacio extra que utiliza para redundar.

3 Códigos de Hamming

Los códigos o bits de Hamming son un conjunto de bits de paridad que se calculan sobre un mensaje y que se intercalan en el mismo de manera que la detección y corrección de errores sea eficiente y además ocupen el mínimo espacio extra posible. Fueron diseñados por Richard W. Hamming en los *Bell Labs* a raíz de notar que constantemente se desechaba largo y costoso cómputo que se realizaba en máquinas que leían tarjetas perforadas. Su trabajo estuvo influenciado por el trabajo del padre de la teoría de la

información *Claude Shannon* con quien compartió oficina.



El aporte de Hamming consiste en calcular bits de paridad sobre porciones de un mensaje tales que la información que proveen de manera combinada permita reducir a una sola posición la locación de un error. Cada bit de paridad funciona como una pregunta de respuesta sí o no y predica sobre un área del mensaje. Para un mensaje (list 0 0 1 1 0 0 0 1 1 1 0) de longitud 11 bits se requieren 15 bits en total.

			0
0	1	2	3
	0	1	1
4	5	6	7
	0	0	0
8	9	10	11
1	1	1	0
12	13	14	15

Los bits que están en posiciones potencias de 2 (1, 2, 4 y 8) los utilizaremos para almacenar paridades al igual que el bit en posición 0 mientras que el resto de los bits serán el mensaje.

El bit que está en la posición 1 es un bit de paridad que se calcula sobre los bits de las columnas impares, esto es, los bits en posiciones 1, 3, 5, 7, 9, 11, 13 y 15.

	0	1	2	3
	0			0
4	0	5	6	7
	0		0	0
8	0	9	10	11
	0		0	0
12	1	13	14	15
	1		0	

El bit que está en la posición 2 se calcula sobre las últimas dos columnas, los bits en posiciones 2, 3, 6, 7, 10, 11, 14 y 15.

	0	1	2	3
	0	1	0	
4	0	1	1	7
	0	0	0	11
12	1	1	1	15

El bit que está en la posición 4 predica sobre las posiciones en filas pares: 4, 5, 6, 7, 12, 13, 14 y 15.

	0	1	2	3
	0	1	0	
1	0	1	1	
	4	5	6	7
	8	9	10	11
0	1	1	0	
	12	13	14	15

Y el bit en posición 8 predica sobre las posiciones 8, 9, 10, 11, 12, 13, 14 y 15.

	0	1	0
	0	1	0
1	0	1	1
1	0	0	0
1	1	1	0

Notar que en todos los casos el bit de paridad forma parte del grupo que se está verificando.

¿Y con el bit en 0 qué hacemos?

3.1 Localización de errores

Si producto de la transmisión o del almacenamiento del mensaje el bloque sufre una corrupción entonces se puede construir un algoritmo que detecta en qué bit se produjo el error. En el ejemplo anterior si se invierte el 1 que está en la posición 6 entonces hay un par de bits de paridad que no están bien calculados.

	0	1	0
	0	1	0
1	0	0	1
1	0	0	0
1	1	1	0

El bit de paridad en posición 1 está bien lo cual descarta que el error se encuentre en columnas impares pero el bit de paridad en posición 2 está mal lo cual indica que el error está en una de las últimas 2 columnas. Ésta información combinada indica que el error se produjo en la tercera columna.

	0	0	1	0	2	3
1	4	0	0	1	6	7
1	8	0	0	0	10	11
1	12	1	1	1	14	15

Un análisis parecido podemos hacer sobre las filas: el bit en posición 4 está mal lo cual obliga al error a estar en una fila par y el bit en posición 8 es correcto lo cual descarta las dos últimas filas. De esto tenemos que el error está en la tercera columna y en la segunda fila y por lo tanto unívocamente determinado.

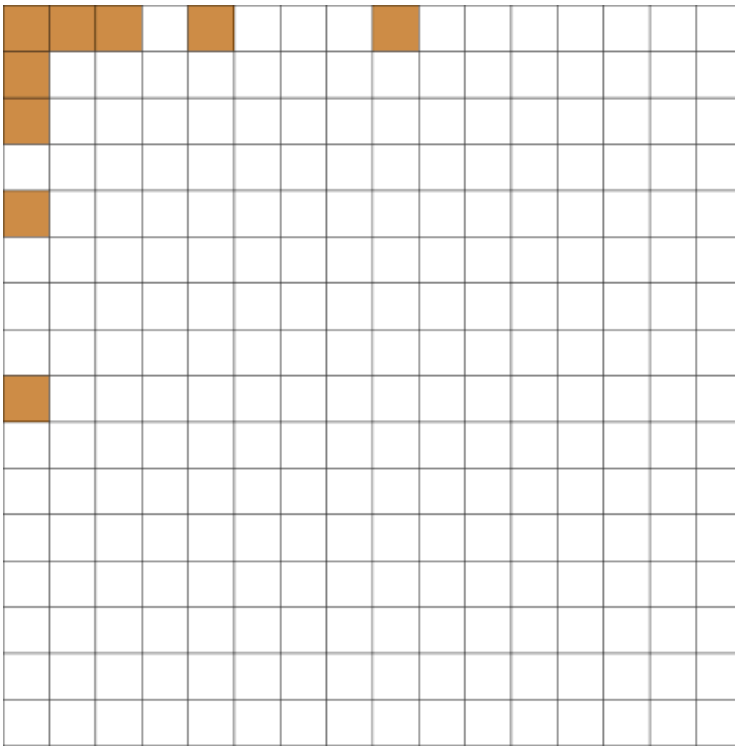
Para asegurarse de que haya comprendido la idea general de cómo detectar errores intente con un error en posición 13.

	0	1	2	3
	0	1	0	
1	4	5	6	7
1	8	9	10	11
1	12	13	14	15

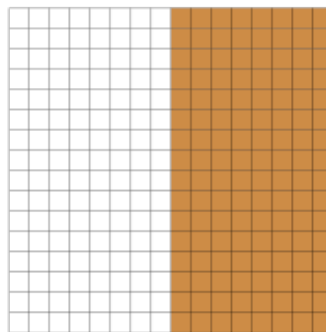
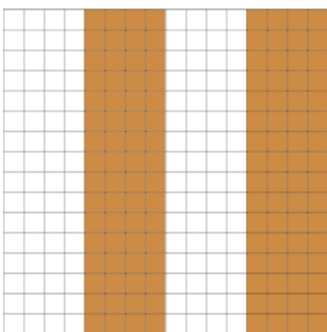
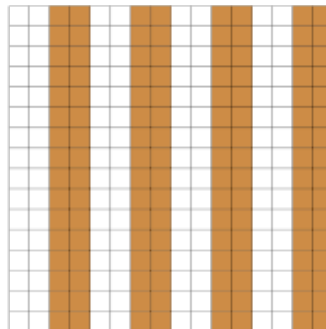
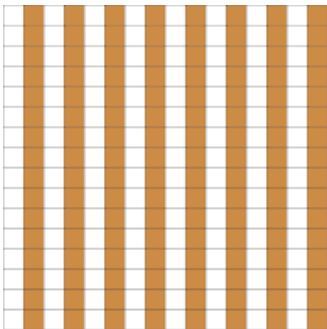
Intente convencerse de que los 4 bits de paridad cubren todo el mensaje. Calcule qué conjunto de bits de paridad deben fallar para exponer cada bit fallido en el mensaje.

Esta construcción se puede generalizar para mensajes de cualquier tamaño. Los bits de paridad seguirán predicando sobre segmentos del mensaje pero crecerán en número mucho más lento que el tamaño del mensaje. Esto es, a medida que crezca el mensaje la cantidad de bits de paridad que se necesiten para "cubrirlo" se incrementará en menor medida.

Por ejemplo, para mensajes de hasta 247 bits alcanzarán 8 bits de paridad (además del 0) para cubrir todo el mensaje.



La mitad de los bits de paridad predicarán sobre "columnas" mientras que la otra mitad predicarán sobre "filas". De aquellos bits que se asocian a columnas notar también cómo están elegidas las columnas para que en la búsqueda la información que provee un bit reduzca los bits candidatos a error a la mitad.



Algo análogo ocurrirá con el cubrimiento de filas. Esto en computación se conoce como búsqueda binaria porque en cada paso descartamos la mitad de los elementos candidatos.

3.2 Cálculo de grupos

De las imágenes de la sección anterior vemos que para un mensaje de 11 bits conviene

disponer los bits en una grilla de 4x4 para visualizar más fácilmente los grupos sobre los que calcular paridad. Aunque esto puede ser útil para números chicos deja de ser práctico a medida que el tamaño del mensaje escala.

En la plantilla cuenta con una función que calcula la cantidad de bits de paridad que se necesitan para un mensaje de tamaño arbitrario. A partir de esta cantidad de bits, en el ejemplo un bloque de 16 bits, nos gustaría poder generar la lista de posiciones asociadas a cada bit de paridad.

```
(check-expect (obtener-grupos 16)
  (list (list 1 3 5 7 9 11 13 15) ; posiciones asociadas al bit de paridad en 1
        (list 2 3 6 7 10 11 14 15) ; posiciones asociadas al bit de paridad en 2
        (list 4 5 6 7 12 13 14 15) ; posiciones asociadas al bit de paridad en 4
        (list 8 9 10 11 12 13 14 15))) ; posiciones asociadas al bit de paridad en 8
```

Ver las posiciones de los bits como se mostraron anteriormente no es tan revelador como verlas en sistema binario.

	0	1	0
0000	0001	0010	0011
1	0	1	1
0100	0101	0110	0111
1	0	0	0
1000	1001	1010	1011
1	1	1	0
1100	1101	1110	1111

3.2.1 Sistema binario

El sistema binario es un sistema de numeración posicional como lo es el decimal. Esto significa que cada símbolo contribuye diferente dependiendo de la posición en la que se encuentra lo cual da lugar a las unidades, decenas, centenas, etc.

Así como en el sistema decimal contamos con 10 dígitos para representar todos los números (0, 1, 2, ..., 9) en el sistema binario contamos solamente con 2 símbolos: 0 y 1. Se cuenta igual que lo hacemos en sistema decimal pero con menos símbolos.

Los primeros 5 números en binario son 0 (0 en decimal), 1 (1 en decimal), 10 (2 en decimal), 11 (3 en decimal) y 100 (4 en decimal). Notar como cada vez que se agotan los símbolos en una posición se agrega un 1 en la posición siguiente y se resetea lo acumulado a 0. Esto pasa de igual manera en el sistema decimal: luego del 59 viene el 60 y luego del 99 viene el 100. Por ejemplo, el número que le sigue al 101 en binario es 110 mientras que el número que se sigue al 111 en binario es el 1000.

En este trabajo vamos a representar números binarios con listas de ceros y unos comenzando con la cifra menos significativa. Esto es el primer caracter de la lista será la cifra correspondiente a las unidades, la siguiente la que corresponde a decenas, luego

vendrán las centenas y así sucesivamente.

Convertir un número natural a uno binario Para convertir un número natural a uno binario tenemos que dividir por 2 el número sucesivas veces, registrar restos y continuar con los cocientes hasta llegar a cociente 1. Por ejemplo, para convertir 6 a binario lo dividimos por 2 y registramos resto y cociente: **0** y **3**. Luego continuamos dividiendo el 3 por dos para obtener resto y cociente **1** y **1** resp. Como el cociente es 1 terminó el algoritmo. Para construir el número en binario basta con leer los restos de las divisiones y el último cociente en orden inverso. Y resulta que 6 en binario es **110**.

Practiquemos con un ejemplo más grande. Nos proponemos convertir 34 a binario.

- $34 = 17 * 2 + 0$ entonces continuamos dividiendo 17 y registramos el resto 0
- $17 = 8 * 2 + 1$ entonces continuamos dividiendo 8 y registramos el resto 1
- $8 = 4 * 2 + 0$ entonces continuamos dividiendo 4 y registramos el resto 0
- $4 = 2 * 2 + 0$ entonces continuamos dividiendo 2 y registramos el resto 0
- $2 = 1 * 2 + 0$ y terminamos con cociente 1.

34 en binario es el último cociente seguido de los restos en orden inverso: **100010**.

Recuerde que en racket al número en sistema binario lo construiremos desde la cifra menos significativa a la cabeza por lo que 34 en binario es `(list 0 1 0 0 0 1)`.

Convierta los naturales 20, 32 y 15 para familiarizarse con el algoritmo.

Ejercicio 2. Diseñe una función `natural->binario` que dado un número natural 'n' devuelva una lista de ceros y unos que represente 'n' en binario comenzando desde la cifra menos significativa.

```
(check-expect (natural->binario 3) (list 1 1))
(check-expect (natural->binario 5) (list 1 0 1))
(check-expect (natural->binario 6) (list 0 1 1))
(check-expect (natural->binario 34) (list 0 1 0 0 0 1))
```

Las funciones de *racket* `remainder` y `quotient` que devuelven el resto y el cociente de hacer una división respectivamente pueden ser útiles.

Note que al poner la cifra menos significativa por delante de la lista se pueden agregar ceros al final de la misma sin alterar el número. Algo parecido ocurre con los números naturales si agregamos ceros delante de un número: 6 y 0006 son el mismo número.

Ahora veamos cómo convertir desde el sistema binario.

Convertir un número binario a uno natural Así como en el sistema decimal el número 2451 no es más que $2 * 10^3 + 4 * 10^2 + 5 * 10^1 + 1 * 10^0 = 2 * 1000 + 4 * 100 + 5 * 10 + 1 * 1$, utilizaremos en el sistema binario una descomposición parecida. En el sistema binario, a diferencia del sistema decimal, la base es 2.

El número **100010** es $1 * 2^5 + 0 * 2^4 + 0 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0 = 1 * 32 + 0 * 16 + 0 * 8 + 0 * 4 + 1 * 2 + 0 * 1 = 32 + 2 = 34$. La descomposición binaria es comparable con la decimal. Las diferencias radican en que

- tenemos disponibles 2 símbolos en vez de 10 lo cual hace que tengamos que prender bits más rápido que en el sistema decimal (cada 10 unidades sumamos 1 a la siguiente posición) y
- las potencias usan base 2 en vez de 10.

Como en el sistema binario solamente contamos con 2 símbolos y uno de ellos es 0, encontrar la descomposición binaria de un número se reduce a sumar las potencias de 2 asociadas a los bits en 1.

Veamos un último ejemplo. El número binario **10011100** (que en racket es `(list 0 0 1 1 1 0 0 1)`) es $1 * 2^7 + 1 * 2^4 + 1 * 2^3 + 1 * 2^2 = 2^7 + 2^4 + 2^3 + 2^2 = 156$.

Convierta los binarios 10100, 100000 y 1111 a números naturales para familiarizarse con el algoritmo.

Ejercicio 3. Diseñe una función `binario->natural` que dada una lista de ceros y unos que representan un número en binario desde la cifra menos significativa devuelva el número natural equivalente.

```
(check-expect (binario->natural (list 1 1)) 3)
(check-expect (binario->natural (list 1 0 1)) 5)
(check-expect (binario->natural (list 0 1 1)) 6)
(check-expect (binario->natural (list 0 1 0 0 0 1)) 34)
```

Ahora que completamos la digresión sobre el sistema binario volvamos a la imagen de los bits de un mensaje con los bits de paridad de Hamming y sus posiciones en sistema binario.

	0	1	0
0000	0001	0010	0011
1	0	1	1
0100	0101	0110	0111
1	0	0	0
1000	1001	1010	1011
1	1	1	0
1100	1101	1110	1111

Veamos cada uno de los grupos de bits sobre los que hay que calcular paridades. ¿Qué tienen en común las posiciones asociadas al bit de paridad 1? ¿Y las posiciones asociadas al bit 2 (10 en binario)?

	0	1	0
0000	0001	0010	0011
1	0	1	1
0100	0101	0110	0111
1	0	0	0
1000	1001	1010	1011
1	1	1	0
1100	1101	1110	1111

Vemos que el grupo de posiciones asociadas al bit de paridad 1 tienen el bit menos significativo en 1.

	0	1	0
0000	0001	0010	0011
1	0	1	1
0100	0101	0110	0111
1	0	0	0
1000	1001	1010	1011
1	1	1	0
1100	1101	1110	1111

Mientras que las posiciones asociadas al bit 2 (10 en binario) tienen el penúltimo bit en 1

	0	1	0
0000	0001	0010	0011
1	0	1	1
0100	0101	0110	0111
1	0	0	0
1000	1001	1010	1011
1	1	1	0
1100	1101	1110	1111

Las posiciones asociadas a 4 (100 en binario) tienen el antepenúltimo bit en 1.

	0	1	0
0000	0001	0010	0011
1	0	1	1
0100	0101	0110	0111
1	0	0	0
1000	1001	1010	1011
1	1	1	0
1100	1101	1110	1111

Y las posiciones asociadas al bit de paridad en 8 (1000 en binario) tienen el primer bit (el más significativo) en 1.

Además vemos que los bits de paridad están en las únicas posiciones que tienen solamente un dígito en 1.

Ejercicio 4. Diseñe una función `uno-en-pos?` que dada una lista de ceros y unos que representan un número binario desde la cifra menos significativa y un número natural 'p' determine si hay un uno en la posición 'p'. Recuerde que un número binario se puede "completar" con ceros al final.

```
(check-expect (uno-en-pos? (list 1 0 0) 0) #true)
(check-expect (uno-en-pos? (list 0 1 0) 1) #true)
(check-expect (uno-en-pos? (list 0 0 1) 2) #true)
(check-expect (uno-en-pos? (list 1 1 1) 3) #false)
(check-expect (uno-en-pos? (list 1 1 1) 100) #false)
```

Ejercicio 5. Diseñe una función `obtener-grupos` que dado el tamaño de bloque de un mensaje con corrección Hamming devuelve una lista de listas de números donde cada lista contiene el conjunto de posiciones asociadas a un bit de paridad. Cada bit de paridad está en su grupo asociado.

```
(check-expect (obtener-grupos 16)
  (list (list 1 3 5 7 9 11 13 15) ; posiciones asociadas al bit de paridad en 1
        (list 2 3 6 7 10 11 14 15) ; posiciones asociadas al bit de paridad en 2
        (list 4 5 6 7 12 13 14 15) ; posiciones asociadas al bit de paridad en 4
        (list 8 9 10 11 12 13 14 15))) ; posiciones asociadas al bit de paridad en 8
```

Para diseñar esta función genere todas las posiciones válidas del 0 hasta el tamaño del bloque para lo cual le puede ser útil la función `posiciones` provista. Luego transforme dichas posiciones a su representación en binario. Y por cada bit de paridad quédese con aquellas posiciones en binario que tienen un uno en el bit que identifica al grupo: último para el bit uno, penúltimo para el bit 2, etc. Finalmente convierta aquellos listados de números en binario de vuelta a números naturales.

3.3 La posición 0

Hasta ahora no hicimos mención de qué ocurre con el bit 0. En principio podríamos

pensar que puede ser un bit útil para transmitir parte del mensaje.

Hasta ahora para un mensaje de 11 bits requerimos 4 bits de paridad. Todas las combinaciones de esos 4 bits de paridad permiten dar con exactamente 1 posición de 16 disponibles en caso de error. Pero no tenemos forma de transmitir que **no** hubo error. Por esa razón dejamos el bit 0 vacío para que si las respuestas a las 4 preguntas son que **no** hubo error en cada uno de los grupos entonces no hubo error en la transmisión del mensaje.

Para no dejar ese bit vacante podemos usarlo como un bit extra de cálculo de paridad. Ese bit puede registrar la paridad de todo el bloque y así proteger los bits de paridad.

¿Quién protege a quién protege?

Como consecuencia de esto más adelante veremos cómo se puede usar este bit de paridad extra para detectar si hubo más de un error en la transmisión.

4 Detección y corrección de errores con códigos de Hamming

Ya tenemos definidas las funciones más interesantes para construir un bloque con corrección de Hamming a partir de un mensaje. Veamos como combinar todo para construirlo y luego usarlo para la corrección de errores.

4.1 Construcción del bloque

Ejercicio 6. Diseñe una función `valores-en-posiciones` que recibe una lista de valores `'lv'` y una lista de posiciones `'ln'` y devuelve una lista con los elementos de `'lv'` que están en las posiciones de `'ln'`.

Puede asumir que la lista de posiciones viene ordenada de forma ascendente.

```
(check-expect (valores-en-posiciones (list 12 21 34 47) (list 0 1 2)) (list 12 21 34))
(check-expect (valores-en-posiciones empty (list 0 1 2)) empty)
(check-expect (valores-en-posiciones (list "a" "b" "c") empty) empty)
(check-expect (valores-en-posiciones (list 1 "d" #true) (list 0 1 3 5)) (list 1 "d"))
```

Ejercicio 7. Diseñe un predicado `es-potencia-de-dos?` que recibe un número y devuelve `#true` si es potencia de dos o 0 y `#false` en otro caso.

```
(check-expect (es-potencia-de-dos? 0) #true)
(check-expect (es-potencia-de-dos? 256) #true)
(check-expect (es-potencia-de-dos? 1) #true)
(check-expect (es-potencia-de-dos? 5) #false)
(check-expect (es-potencia-de-dos? 6) #false)
```

Ejercicio 8. Diseñe una función `ubicar-mensaje` que recibe un mensaje y una lista de números desde el 0 hasta el tamaño de bloque y devuelve el bloque relleno con el mensaje. Además settea las posiciones correspondientes a los bits de corrección (potencias de dos o cero) en el bloque a 0.

```
(check-expect (ubicar-mensaje (list 1 0 1 0 0 1 0 1 0 0 1)
                              (range 0 16 1))
```

```
(list 0 0 0 1 0 0 1 0 0 0 1 0 1 0 0 1))
```

Las posiciones en donde van bits de paridad se settean a cero para no alterar la paridad en cálculos posteriores.

El siguiente paso para construir el bloque y posiblemente el más importante es calcular las paridades y colocarlas en las posiciones correspondientes en el bloque.

Ejercicio 9. Diseñe una función `calcular-paridades` que recibe una lista de listas de posiciones 'grupos' y un bloque que contiene un mensaje y devuelve una lista de paridades asociadas a cada grupo y calculadas sobre el bloque.

```
(check-expect (calcular-paridades (obtener-grupos 16)
                                   (list 0 0 0 0
                                         0 0 1 1
                                         0 0 0 0
                                         1 1 1 0))
              (list 0 1 1 1))
(check-expect (calcular-paridades (obtener-grupos 16)
                                   (list 0 0 0 1
                                         0 0 1 0
                                         0 0 1 0
                                         1 0 0 1))
              (list 0 0 1 1))
```

Utilice las definiciones de paridad y de valores-en-posiciones para definirla.

Ahora pensemos cómo **ubicar los bits de paridad** en el bloque.

Como los bits de paridad van en posiciones potencias de 2 o cero, utilizaremos una técnica conocida como *tupling* (emparejamiento) para poder llevar registro de los bits que hay en un bloque y a la vez su posición.

Definimos en racket la estructura `Par`.

```
(define-struct Par [p s])
; Par es un (Any, Any)
; p es la primera componente del par
; s es la segunda componente del par
```

Un `Par` es una estructura que nos permite emparejar elementos de diferente tipo.

Para poder calcular e insertar las paridades partiremos de una lista de ceros y unos, esto es, el bloque con el mensaje. Nos interesa llegar a una lista de pares donde las primeras componentes de los pares son los bits del bloque y las segundas componentes son las posiciones de los bits.

```
(emparejar (list 0 0 0 1
                0 0 1 0
                0 0 1 0
                1 0 0 1)
           (range 0 16 1))

== (list
    (make-Par 0 0)
    (make-Par 0 1)
```

```

(make-Par 0 2)
(make-Par 1 3)
(make-Par 0 4)
(make-Par 0 5)
(make-Par 1 6)
(make-Par 0 7)
(make-Par 0 8)
(make-Par 0 9)
(make-Par 1 10)
(make-Par 0 11)
(make-Par 1 12)
(make-Par 0 13)
(make-Par 0 14)
(make-Par 1 15))

```

Ejercicio 10. Diseñe una función emparejar que recibe un par de listas y devuelve una lista de pares donde las primeras componentes son elementos de la primera lista y las segundas componentes son elementos de la segunda. La lista resultante tiene el largo de la lista más corta.

emparejar : List (X) -> List (Y) -> List (Par [X ,Y])

Ejercicio 11. Diseñe una función ubicar-paridades que recibe una lista de paridades (incluida la paridad del bloque completo para la posición 0) y un bloque con mensaje y devuelve un bloque con las paridades agregadas en las posiciones potencias de 2 y cero.

```

(check-expect (ubicar-paridades (list 0 1 1 1 0)
                                (list 0 0 0 1
                                      0 0 1 0
                                      0 0 1 0
                                      1 0 0 1)))

(list 0 1 1 1
      1 0 1 0
      0 0 1 0
      1 0 0 1))

```

Para implementar esta función primero empareje los bits del bloque con las posiciones del 0 al tamaño del bloque. Luego recorra esa lista y en caso de estar en una posición potencia de dos o 0 reemplace el bit de paridad por el correspondiente en la lista de paridades, caso contrario deje el bit del mensaje original.

Ejercicio 12. Diseñe una función mensaje-a-bloque que recibe un mensaje y devuelve un bloque con mensaje y códigos de corrección de Hamming.

```

(check-expect (mensaje-a-bloque (list 0
                                       0 1 1
                                       0 0 0
                                       1 1 1 0)))

(list 0 0 1 0
      1 0 1 1
      1 0 0 0
      1 1 1 0))

```

Para definir esta función utilice ubicar-mensaje, calcular-paridades y ubicar-paridades, calcule la paridad del bit en 0 y ubíquelo como primer bit. Agregue otros casos de prueba a la función.

4.2 Análisis del bloque

Luego de que se crea un bloque con bits de paridad de Hamming, producto de la transmisión del bloque o de su almacenamiento puede ocurrir que algún bit haya cambiado. Por un momento dejemos fuera del análisis qué ocurre con el bit 0, luego lo retomaremos.

La forma más directa de detectar un error es verificar por cada bit de paridad si el grupo de posiciones asociado a cada paridad se corresponde con la paridad del grupo de bits. En caso de que exista una falla podemos identificar unívocamente el bit en donde se produjo: cada bit de paridad está asociado a un conjunto de posiciones, si el bit no coincide en paridad con el grupo entonces se considera ese grupo defectuoso, caso contrario su complemento. La intersección de todos esos conjuntos resulta en una sola posición que es donde se produjo el error.

Esta forma directa de resolver errores con códigos de Hamming aunque correcta puede resultar un poco costosa porque involucra generar los conjuntos de posiciones tal cual lo hicimos en la anterior sección, determinar complementos de conjuntos de posiciones para los bits de paridad que sean correctos y calcular la intersección de unas cuantas listas.

Seguiremos otro camino para la corrección.

	0	1	0
1	0	0	1
1	0	0	0
1	1	1	0

Si se produce un error en la posición 6, 0110 en binario, eso significa que el bit en la posición 1 sigue correcto, los bits en posiciones 2 y 4 están incorrectos y el bit en la posición 8 está correcto. Veamos ahora las posiciones en binario.

	0	1	0
0000	0001	0010	0011
1	0	0	1
0100	0101	0110	0111
1	0	0	0
1000	1001	1010	1011
1	1	1	0
1100	1101	1110	1111

Si nombramos los bits de paridad en orden inverso y registramos si están correctos con un 0 o están incorrectos con un 1 vemos que la secuencia de ceros y unos que nos queda es 0110. Los ceros y unos obtenidos responden a la pregunta de si **hay error en el bit de paridad** para cada bit de paridad (traduciendo `#true` a 1 y `#false` a 0). Veamos que la posición 6 no una posición privilegiada respecto a las otras posiciones y justifiquemos por qué los bits de paridad se leen en orden inverso.

	0	1	0
0000	0001	0010	0011
1	0	1	1
0100	0101	0110	0111
1	0	0	0
1000	1001	1010	1011
1	0	1	0
1100	1101	1110	1111

Si se produce un error en el bit 13, 1101 en binario, entonces los bits de paridad que son erróneos son 1, 4 y 8. Respondamos a la pregunta de si hay error en el bit de paridad de atrás hacia adelante en el listado de bits de paridad.

- Hay error en el bit 8 (1, `#true`)
- Hay error en el bit 4 (1, `#true`)
- No hay error en el bit 2 (0, `#false`)
- Hay error en el bit 1 (1, `#true`)

Concluimos que verificar si la paridad es correcta para el listado de bits de paridad de atrás hacia adelante construye la posición del bit con error en el bloque. Como vimos en la sección anterior, cada bit de paridad contribuye un bit a la posición del error, en caso de existir error.

Hagamos algunas observaciones más antes de implementar esta solución.

Operación XOR Repasemos primero la operación binaria **xor** u *or* exclusivo. El *or* exclusivo devuelve 1 (o `#true`) sólo si exclusivamente una de sus entradas es 1 (o `#true`).

a	b	a xor b
0	0	0
0	1	1
1	0	1
1	1	0

Alternativamente se puede pensar a la operación como una suma módulo 2 (resto de la división por dos). Esta última interpretación nos permite pensar en que el xor calcula las paridades de los bits de entrada.

También podemos pensar la operación extendida que recibe listas de ceros y unos y calcula el xor bit a bit.

```
(check-expect (xorL (list 0 0 1 1 1 0 1 0 1)
                    (list 0 1 0 1 0 1 1 1 1))
              (list 0 1 1 0 1 1 0 1 0))
```

Más aún, si hacemos el xorL de más de un par de listas entonces estamos computando la paridad "por columna", es decir, calculamos la paridad del conjunto de bits que están en la posición 0, luego la paridad de los bits que están en posición 1 y así sucesivamente.

Para usar la operación xor en racket tenemos que incluir la librería bool. Eso se puede hacer con la línea `(require racket/bool)`

Volvamos a detectar los errores con corrección de Hamming. Pensemos qué ocurre si aplicamos la operación **xor** a las posiciones donde hay unos en el bloque.

	0 0000	1 0001	0 0010	0 0011
1 0100	0 0101	1 0110	1 0111	
1 1000	0 1001	0 1010	0 1011	
1 1100	0 1101	1 1110	0 1111	

El xor de los últimos bits de las posiciones en donde hay unos cuenta cuántos unos hay en posiciones asociadas al bit de paridad en 1.

	0 0001		0 0011
	0 0101		1 0111
	0 1001		0 1011
	1 1101		0 1111

El xor de los penúltimos bits de las posiciones en donde hay unos cuenta cuántos unos hay en posiciones asociadas al bit de paridad en 2.

		1 0010	0 0011
		1 0110	1 0111
		0 1010	0 1011
		1 1110	0 1111

El xor de los segundos bits de las posiciones en donde hay unos cuenta cuántos unos hay en posiciones asociadas al bit de paridad en 4.

1 0100	0 0101	1 0110	1 0111
1 1100	1 1101	1 1110	0 1111

El xor de los primeros bits de las posiciones en donde hay unos cuenta cuántos unos hay en posiciones asociadas al bit de paridad en 8.

1 1000	0 1001	0 1010	0 1011
1 1100	1 1101	1 1110	0 1111

Entonces hacer el **xor** de todas las posiciones en binario asociadas al bloque con valor 1 equivale a hacer todos los chequeos de paridad. En el ejemplo, el xor de las siguientes posiciones (posiciones de unos en el bloque)

```

0  0  1  0
0  1  0  0
0  1  1  0
0  1  1  1
1  0  0  0
1  1  0  0
1  1  1  0

```

resulta en **1101**. Precisamente la posición del error que habíamos detectado.

	0 0001	1 0010	0 0011
1 0100	0 0101	1 0110	1 0111
1 1000	0 1001	0 1010	0 1011
1 1100	0 1101	1 1110	0 1111

¿Qué ocurre si no hay errores? Analicemos qué ocurre con el xor de las posiciones del bloque con unos si no hay errores.

	0	1	0
0000	0001	0010	0011
1	0	1	1
0100	0101	0110	0111
1	0	0	0
1000	1001	1010	1011
1	1	1	0
1100	1101	1110	1111

Las posiciones con unos son las que siguen.

```

0  0  1  0
0  1  0  0
0  1  1  0
0  1  1  1
1  0  0  0
1  1  0  0
1  1  0  1
1  1  1  0

```

Y su xor resulta en **0000**.

¿De qué manera un bit erróneo cambia el resultado del xor? Partiendo del caso en que no hay errores, invertir un bit puede producir que o bien se agregue una posición al xor (cambia un 0 por un 1) o bien que se remueva una posición al xor (cambia un 1 por un 0).

- Si cambia un 0 por un 1 entonces se agrega una posición a la lista de posiciones sobre las que hacer el xor. Esa nueva posición que se agrega (que se corresponde con el bit erróneo) resulta en que el xor devuelve esa nueva posición. Esto se justifica porque el neutro de la operación xor es 0: $(\text{xor } 0 \text{ } n) == n$ para todo n . Si partimos de un bloque sin errores (cuyo xor de posiciones con unos es 0) e introducimos un error cambiando de 0 a 1 un bit entonces el resultado del xor es la posición errónea.
- Si se cambia un 1 por un 0 entonces tenemos que remover del xor la posición del bit invertido. Otra propiedad de la operación xor es que al operar un elemento consigo mismo obtenemos el neutro, 0. Esto nos dice que remover una posición del xor es equivalente a agregar una vez más dicha posición pues el xor de dos posiciones iguales se cancela. Y esto justifica por qué invertir un 1 por un 0 (también) resulta en que el xor indica la posición del bit erróneo.

Ahora nos proponemos implementar todo lo discutido en la última sección.

Ejercicio 13. Diseñe una función `xorL` que recibe un par de listas y devuelve el resultado de aplicar xor bit a bit.

Como utilizaremos `xorL` sobre listas de números ceros y unos que representan posiciones en binario desde bit menos significativo debemos **completar las listas** para que tengan el

mismo largo. Esto lo hacemos **con ceros al final** de las mismas ya que los ceros al final de la lista (al principio del número en binario) no alteran el número. Sírvese de los ejemplos para implementarla correctamente.

```
(check-expect (xorL (list 0 1 0 1)
                    (list 0 0 0 0))
              (list 0 1 0 1))
(check-expect (xorL (list 0 0 1 1)
                    (list 0 0 1 1))
              (list 0 0 0 0))
(check-expect (xorL (list 0 0 1 1)
                    (list 0 1 0 1))
              (list 0 1 1 0))
(check-expect (xorL (list 0 0 1 1)
                    (list 0 1 0 1 1))
              (list 0 1 1 0 1))
(check-expect (xorL (list 0 0 1 1 0)
                    (list 0 1 0 1))
              (list 0 1 1 0 0))
```

Ejercicio 14. Diseñe una función detectar-error que recibe un bloque de bits, realiza el xor de las posiciones en binario que contienen 1 y devuelve la posición en donde se detectó un error como número natural.

Para esto volveremos a usar la técnica de *tupling* o emparejamiento. Al bloque de bits lo emparejamos con una lista de posiciones desde el 0 al tamaño del bloque para obtener una lista de pares. Luego nos quedamos con aquellos pares cuyos bits de bloque tienen el valor 1. A continuación convertimos a binario dichas posiciones y las operamos con xorL. Finalmente convertimos el número binario resultante (lista de ceros y unos) a un número natural.

Agregue los ejemplos vistos en la sección como casos de prueba.

Ejercicio 15. Diseñe un predicado bit-0-correcto? que recibe un bloque de bits y devuelve #true en el caso que el bit en posición 0 indique la paridad correcta del bloque.

```
(check-expect (bit-0-correcto? (list 0
                                    0 1 1 0))
              #true)
(check-expect (bit-0-correcto? (list 1
                                    0 1 1 0))
              #false)
(check-expect (bit-0-correcto? (list 0
                                    0 1 1 1))
              #false)
(check-expect (bit-0-correcto? (list 1
                                    0 1 1 1))
              #true)
```

Terminamos este trabajo con una función mensaje-error que recibe el bloque de bits transmitido y detecta si hubo errores. Analicemos con cuáles casos nos encontramos.

- Si detectar-error devuelve 0 y el bit de paridad en cero es correcto entonces no hubo bits cambiados en la transmisión del mensaje.
- Si detectar-error encontró una posición con error (distinta de cero) y el bit de

paridad en cero es incorrecto entonces la posición que encontró detectar-error es la única en donde se produjo un error. En este caso podemos arreglar el error invirtiendo dicho bit.

- Si detectar-error encontró una posición con error pero el bit de paridad es correcto significa que se produjo más de una inversión en la transmisión. Este caso lo podemos detectar pero no lo podemos corregir.

Ejercicio 16. Diseñe una función mensaje-error que recibe un bloque de bits y devuelve un string que indica si hubo errores. En caso de que el bloque tenga exactamente un error, la función indica la posición.

Los resultados posibles de la función son "No hay errores", "Hay exactamente un error en X" donde X se reemplaza por la posición en donde ocurrió un error o "Hay más de un error".

Utilice detectar-error y bit-0-correcto? para definirla.

```
(check-expect (mensaje-error (list 0 0 1 0
                                   1 0 1 1
                                   1 0 0 0
                                   1 1 1 0)))
               "No hay errores")
(check-expect (mensaje-error (list 0 1 1 0
                                   1 0 1 1
                                   1 0 0 0
                                   1 1 1 0)))
               "Hay exactamente un error en 1")
(check-expect (mensaje-error (list 0 0 0 0
                                   1 0 1 1
                                   1 0 0 0
                                   1 1 1 0)))
               "Hay exactamente un error en 2")
(check-expect (mensaje-error (list 0 0 1 0
                                   1 0 1 0
                                   1 0 0 0
                                   1 1 1 0)))
               "Hay exactamente un error en 7")
(check-expect (mensaje-error (list 0 0 1 0
                                   1 0 1 1
                                   1 0 0 0
                                   0 1 1 0)))
               "Hay exactamente un error en 12")
(check-expect (mensaje-error (list 0 0 1 1
                                   1 1 1 1
                                   1 0 0 0
                                   1 1 1 0)))
               "Hay más de un error")
(check-expect (mensaje-error (list 0 1 1 0
                                   1 0 1 1
                                   1 0 0 0
                                   1 0 1 0)))
               "Hay más de un error")
```

5 Notas

En la práctica es común definir un tamaño máximo de mensaje para transmitir. Si se quiere transmitir un mensaje más largo que el tamaño preestablecido entonces dicho mensaje se segmenta y cada segmento se protege de corrupción con códigos de Hamming. Luego en el receptor cada segmento se chequea por separado y en caso de no poder corregirse un error se solicita retransmisión del segmento (y no de todo el mensaje).

Ejercicio 17. (opcional) Piense cómo agregaría esta funcionalidad de segmentar un mensaje, luego proteger cada segmento con códigos de Hamming y finalmente reconstruir el mensaje en el receptor. Agréguelo en comentarios si desea.